

Herald



Herald — Fixed

Field	Value
Revision	14
Created	2026-05-20
Last modified	2026-05-22
Status	active
Status summary	r14 captures Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure). HRD-100 closed atomically with 12 Wave-6 commit SHAs + 8 new e2e invariants E63-E70 (currently SKIP-with-documented-reason — converting to PASS once operator runs the live closed-loop at T10b and commits docs/qa/HRD-100/). Tag v0.4.0 DEFERRED to T13b (operator-supplied live evidence per §107.x — code-doc closure ships in this commit; runtime evidence lands separately). Wave 6 surfaces: new Cobra subcommand <code>pherald listen</code> (Telegram <code>getUpdates</code> long-poll + <code>OnText/OnPhoto/OnDocument/OnVoice</code> + bot self-filter + sha256 content-addressed attachment download); Opus model pin in spawned <code>claude</code> <code>argv</code> (<code>--model claude-opus-4-7</code>); envelope pre-text — verbatim operator wording "We have received new message from our communication channel" preceding the <code><<<HERALD-DISPATCH-v1>>></code> structured block; <code><<<HERALD-REPLY>>></code> parser with action routing (<code>reply / issue.open / event.emit</code>); <code>tgram.Adapter.SendReply</code> wiring <code>reply_to_message_id</code>; <code>--qa-out-dir</code> JSONL journaling per §107.x. Wave 6 mutation gate (<code>tests/test_wave6_mutation_meta.sh</code>) lands 3 paired hermetic mutations (M1 blank bot self-filter → echo-loop signature; M2 Opus → Sonnet model swap → E67 FAIL; M3 drop <code>opts.ReplyTo</code> in <code>SendReply</code> → E68 FAIL). Prior r13 captured Wave 4b (TOON application/ toon content negotiation; tag v0.3.0). r12 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3; tag v0.2.0). r11 closed HRD-016 (Wave 3b pherald Runner live). r10 closed HRD-011 (Telegram live).
Issues	see <code>Issues.md</code> — r15 captures HRD-100 Issues→Fixed atomic migration.
Issues summary	HRD-008/-015/-018 (in progress) + HRD-019..HRD-027 + HRD-029..HRD-056 + HRD-081 + HRD-085..HRD-090 still open (47 items).
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-028, HRD-080, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099,

Field	Value
Fixed summary	HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114 (and HRD-018 partial — M1 + M2 components landed) spec V1→V3 r9; Go module foundation + Foundation M1/M2/M3; HRD-010 commons_storage live wiring; HRD-011 Telegram live (live_message_id=5 evidence 2026-05-22); HRD-012 Claude Code dispatcher live; HRD-016 pherald /v1/events Runner wiring (Wave 3b live — 7-stage pipeline + e2e E37-E42); universal §11.4.73 + §11.4.74 mandates propagated; I6 gate refined; Wave 2 flavor scaffolds (HRD-092..097) closed atomically; Wave 3a substrate (HRD-099 commons_auth JWT verifier) + 2 live REST routes (HRD-028, HRD-098) closed atomically.

Continuation see CONTINUATION.md.

Table of contents

- [Recently fixed](#)

Recently fixed

ID	Type	Criticality	Title
HRD-114	task	high	Wave 7 T5 — multi-channel pherald_listen (HERALD_CHANNELS fan-f Subscribe goroutines → 1 dispatcher). pherald/internal/inbound/dispatcher.go renames the Wave-6 TgramReplier interface (Telegram-n chatID/int_replyToID signature) → generic inbound.Replier with SendReply(ctx, recipient commons.Recipient, body string, replyToID string, attachments []commons.Attachment) (s error) — recipient + string ids let any channel satisfy it. Config.TgramRep Config.Reply; NewDispatcher nil-check + Dispatcher.reply field Handle "reply" branch + both fastPath branches all migrated to build a commons.Recipient{Channel, ChannelUserID} + strconv.Itoa(replyToID) (" " when 0). pherald/cmd/pherald/li listenConfig.Subscriber (singular) → Subscribers map[string]func(ctx, InboundHandler) error; listenConfig inbound.TgramReplier → inbound.Replier; new loadEnabledCh parses HERALD_CHANNELS (comma-split, trim, dedupe; unset/empty → ["tgr Wave-6 single-channel behaviour preserved); new perChannelConfig(name namespaced env (HERALD_TGRAM_BOT_TOKEN/HERALD_TGRAM_CHAT_ID HERALD_SLACK_BOT_TOKEN/_APP_TOKEN/_CHANNEL_ID) → channel (fail-loud on missing required creds); loadListenConfigFromEnv resolves enabled channel via the channels.New(name, cfg) registry (T2), builds Subscribers[name]=ch.Subscribe + a channelReplier{ch} shim channels.Channel.SendReplyGeneric → inbound.Replier.Send the two intentionally-differently-named generic methods bridged) per channel, and them in a channelRouter that dispatches each reply to the adapter for recipient.Channel (fail-loud no replier for channel %q on unk never silently dropped). runListen fans in one goroutine per subscriber (manu sync.WaitGroup + buffered error channel; no new dep — errgroup-equivalen all feeding ONE dispatcher; the first non-cancel subscriber error cancels siblings (T11 chaos fail-loud). tgram is blank-imported in listen.go for its init() registrat registration. journal.go journalingReplier migrated to wrap inbound.Replier (records channel+chat_id+reply_to_message_

ID

Type Criticality Title

HRD-113 task high

strings; JSONL kind kept `tgram.send_reply` for transcript back-compat). M resolution (Wave 7 plan Step 4): `inbound.Replier.SendReply` keeps the `SendReply` name (the inbound package's own, independent of `channels.Channel.SendReplyGeneric`); production wiring bridges via `channelReplier` shim so the journal decorator + dispatcher both stay `channel`. §107 evidence: new `TestRunListenMultiChannelFanIn` (2 stub subscribers: `tgram+slack` each publish 1 event) asserts BOTH messages dispatched AND both recorded (seen["ack (fake)"] >= 2) — NOT "both goroutines started"; proven load-bearing (dropping the fan-in loop → seen == map[] → FAIL). E single-channel `TestListenWiresHandlerAndExitsOnCancel` + `TestRunListenRejectsBadConfig` migrated to the one-entry `Subscriber` still PASS. `recordingReplier/stubReplierJournal` stubs (`dispatcher_listen_test/journal_test`) migrated to the generic signature; numeric `chat_id/reply` preserved via `decode.go build ./pherald/... + go test -race -count=1 ./pherald/... ./commons_messaging/...` all green; go clean; §107.y quiescence clean.

Wave 7 T4 — generalize the bot self-filter via `BotSelfIdentity` (channel-agnostic) file `commons_messaging/channels/selffilter.go` adds the `Raw-map` constants (`RawSenderIsBot/RawSenderIdentityKind/RawSenderIdentityKind`) `StampSender(raw map[string]any, isBot bool, kind IdentityKind, value string)` (records the sender's native identity into `ev.Raw`; nil-map no `IsSelfEcho(ev commons.InboundEvent, self SelfIdentity)` §32.9 anti-echo-loop guarantee made channel-agnostic — compares the right native `IdentityKind`: `username` for Telegram, `user_id` for Slack, `address` for Scope is deliberately narrow: a DIFFERENT bot is KEPT (multi-bot collaboration subscriber traffic — the §107 anchor that the filter is not over-broad); a human is empty `self.Value` never echoes (conservative KEEP → surfaces as duplicate silent loss; the real guard is `Subscribe` refusing to boot on empty `self`). `commons_messaging/channels/tgram/subscribe.go` migrated: cap `self := channels.SelfIdentity{Kind: channels.IdentityUser, Value: bot.Me.Username}` once at `Subscribe` boot (keeping the existing `ev` username boot-refusal echo-loop guard) and routes ALL FOUR live handlers (`OnPhoto/OnDocument/OnVoice`) through a new `stampAndIsSelfEcho(self)` helper that stamps the Telegram sender's `IsBot+@username` into `ev`. delegates to `channels.IsSelfEcho` — replacing the four Wave-6 `shouldDropBotSelf(msg, selfUsername)` early-returns. The Wave-6-`shouldDropBotSelf` func + its `TestSubscribeBotSelfFilter` + `SelfFilterForTest` (`export_test.go`) are PRESERVED BYTE-IDENTICAL M1 paired-mutation gate (`tests/test_wave6_mutation_meta.sh`) anchored exact-text `perl -i -0pe` regex on `shouldDropBotSelf`'s body, so keeping identical keeps M1 load-bearing (verified: the M1 regex still matches + applies its marker cleanly post-migration). The live path and the gate now express the SAME invariant — production on the generalized `IsSelfEcho`, the gate on the `tgram-§107` evidence: `TestIsSelfEchoUsername` proves all three username cases DROPPED, different-bot KEPT, human KEPT; `TestIsSelfEchoUserID` proves Slack `user_id` kind (matching bot id DROPPED, different id KEPT); `TestIsSelfEchoEmptySelfNeverEchoes` proves empty-self KEEP; the helper builds events via the production `StampSender` (DRY — the test can never break the wire-key contract); `TestSubscribeBotSelfFilter` still PASSES (Wave preserved); `go test -race -count=1 ./commons_messaging/... ./pherald/...` all green; go vet clean. Live runtime evidence (real Telegram identity + own-message drop in the closed loop) lands via Wave 7 T9 e2e invariant

ID

Type Criticality Title

HRD-112 task middle

Wave 7 T3 — per-channel inbox subdirs `~/ .herald/inbox/<channel>/<sha256>.<ext>`. New file `commons_messaging/channels/inbox.channel-agnostic InboxDir(channel string) (string, error) (res` `~/ .herald/inbox/<channel>/, MkdirAll 0700, empty-channel error) + WriteContentAddressed(channel, mime string, r io.ReadC` `(path, sha256Hex string, err error) (streams r into <dir>/<sha256>.<ext> via io.MultiWriter(tmp, sha256.New()) — new the full payload — then atomic temp+rename; idempotent: if the content-addressed already exists the .part temp is dropped and the existing path returned unchanged + MimeToExt(mime string) (the Wave-6 tgram-private mimeToExt switch VERBATIM to package level so every adapter shares one map). commons_mes channels/tgram/attachments.go DownloadAttachment now does Telegram-specific bot.File(&telebot.File{FileID}) fetch then deleg stream-hash-rename to channels.WriteContentAddressed(string(common ChannelTelegram, mime, rc) — so tgram attachments land under ~/ .herald/inbox/tgram channel subdir, was the flat ~/ .herald/inbox/ in Wave 6). The tgram-private MimeToExt + the crypto/sha256/encoding/hex/io/os/path/filepath were removed from attachments.go (the helper owns them now); the method (*Adapter).DownloadAttachment(T1) delegates to the package func un both forms route through the shared helper. §107 evidence: TestInboxDirIsPerChannel proves InboxDir("tgram") ends in .. tgram AND InboxDir("tgram") != InboxDir("slack") (no flat-in regression); TestWriteContentAddressedHashesAndIsIdempotent three §107 bluff classes on real on-disk state — (a) path shape under inbox/sl <sha>.txt, (b) re-read on-disk bytes byte-equal the payload (sha in path == sh content), (c) 2nd write of same content returns same (path, sum) and leaves zero residue (real os.ReadDir scan); the existing Wave-6 TestDownloadAttachmentContentAddressed (updated to the /tgram segment) still PASSES end-to-end against a httptest Bot API, proving the route is behaviour-preserving. go test -race -count=1 ./ commons_messaging/... ./pherald/... ./qaherald/... all gre assumption regressions); go vet clean. Live runtime evidence (real Telegram at landing under inbox/tgram/) lands via Wave 7 T9 e2e invariant E83.`

HRD-111 task high

Wave 7 T2 — channel registry (`name→constructor`) + `init()`-based registration `commons_messaging/channels/registry.go` adds `Register(name Constructor) / New(name, Config) (Channel, error) / Names([]string, the ErrUnknownChannel sentinel, the channel-agnostic Config (Channel/Token/AppToken/Target/BaseURL/Extra), and the Construc func(Config) (Channel, error) type — a sync.RWMutex-guarded map[string]Constructor. Register panics on duplicate (a typo in two init() would silently shadow one — a §107 bluff class this framework forbids qaherald scenario. Register pattern). New wraps ErrUnknownChannel (fmt.Errorf("%w: ...")) for unknown names — callers (pherald lis MUST surface it; a silent no-op channel is a §107 PASS-bluff. commons_mess channels/tgram/tgram.go gains an init() that calls channels.Register(string(common ChannelTelegram), ... constructor builds NewAdapterWithBaseURL(cfg.Token, cfg.Target cfg.BaseURL) when cfg.BaseURL != "" (httptest seam) else NewWithCreds(cfg.Token, cfg.Target) — so a blank import self-reg adapter. §107 evidence: TestRegistryResolvesRegisteredChannel p actually constructs + returns a working Channel (c.Name()=="fake-rt",`

ID

Type Criticality Title

HRD-110 task high

TestRegistryUnknownChannelErrors proves errors.Is(err, ErrUnknownChannel) for unknown names; TestRegistryDuplicateP proves the duplicate-Register panic; TestTgramRegisteredViaInit (t _ ".../tgram") proves the init() actually registered tgram in Names() op — verified failing before the init() landed: Names()=[dup-rt fake-tgram, PASS after). The shared fakeChannel test stub uses SendReplyGeneric T1-resolved interface method name, NOT SendReply). go test -race ./commons_messaging/... green (tgram regression-free); all 16 workspace modules build; go vet ./commons_messaging/... clean. Live runtime evidence (channel resolved by name + round-trip) lands via Wave 7 T9 e2e invariants E81/E82. Wave 7 T1 — extract commons_messaging/channels.Channel interface (refactor). New package commons_messaging/channels/ defines Channel interface the §11.0 commons.Channel — Name/Capabilities/Send/Subscribe/HealthCheck the three inbound-runtime methods Telegram implicitly defined: SendReplyGeneric string-typed reply, BotSelfIdentity returning SelfIdentity{Kind, Value, DownloadAttachment), the SelfIdentity value type, and the Identity constants (IdentityUsername/IdentityUserID/IdentityAddress). *tgram.Adapter gains BotSelfIdentity (getMe via ensureBot; empty-union §107 echo-loop-hazard error), SendReplyGeneric (string→int64/int adapter native Wave-6 SendReply), and a method-form DownloadAttachment (round-trip package-level func through ensureBot's live bot) — so var _ channels.Channel (*tgram.Adapter)(nil) compiles. Naming divergence from plan Step 4 to + documented in channel.go package doc: the interface reply method is named SendReplyGeneric (not SendReply) so tgram satisfies it WITHOUT touching native, §107-mutation-gate-anchored int64/int SendReply (whose migration to signature is T5's scope) — keeping T1 a pure refactor with zero cross-package change evidence: var _ channels.Channel = (*tgram.Adapter)(nil) compiles. assertion in channel_test.go (a renamed method / drifted signature breaks t TestTgramSatisfiesChannel + TestSelfIdentityType PASS; full -race ./commons_messaging/... green (tgram regression-free — pure-proof); all 16 workspace modules build; go vet ./commons_messaging/... Live runtime evidence (real getMe self-identity + reply round-trip) lands via Wave 7 T9 e2e invariant E81.

HRD-101 task high

Wave 6.5 ticket-lifecycle layer — §32.6 deterministic classifier (Bug:/Issue:/Implementation:/Investigation:/Query:/Question:/Help:/Status:/Continue:/Done:/Resolve:/Reopen:/Override: prefix routing with Type+Criticality+Confidence scores; natural-language no-prefix → query fallback) + DocsIssueOpener (§32.9 — real docs/Issues.md HRD-NNN allocated concurrent-safe via lockfile) + RunnerEventEmitter (§33.7 — direct runner.Run CloudEvent dispatch, no HTTP loopback) + Help:/Status:/Continue: fast handlers (bypass Claude Code, sub-second latency — S2 proved 0.74s, no CC bootstrap overhead) + Done:/Reopen: atomic Issues[?]Fixed migration (rollback-via-sha256) tgram.SendReply outbound attachment fan-out (multipart dispatch) + wiring into pherald listen. §107.x live evidence (docs/qa/HRD-101-lifecycle-2026-05-23T03-16-17-w6.5live/): S1 natural-language real Opus CC dispatch ~31s incl. bootstrap, threaded reply (wire-byte proof inbound message_id=30 → reply sent_message_id=31 with reply_to_message_id=30); S2 Help: fast-path — classifier Type: help Confidence: 1, CC dispatch ABSENT (event absence + 0.74s latency prove no BuiltinHelp catalogue text returned (reply_to_message_id=32 → sent_message_id=33); transcript.jsonl captures both directions of all four messages query, Help:, Status: also exercised live). Reproducible test harness tests/

ID

Type Criticality Title

test_wave6.5_lifecycle.sh covers 15 scenarios (Bug:/Task:/Query: + a Done:/Reopen: + classifier edge cases). Spec V3 r13: §32.6/§32.9/§33.6/§33.7 su landed. Tag v0.5.0.

Wave 6 — pherald inbound runtime + Claude Code headless bridge (code-doc cl Cobra subcommand pherald listen runs the closed-loop runtime: Telegram getUpdates long-poll (OnText/OnPhoto/OnDocument/OnVoice) + bot s (bot.Me.Username anti-echo guard captured at Subscribe construction; refuse getMe returned no username) + sha256 content-addressed attachment download ~/.herald/inbox/<sha256>.<ext> (idempotent — duplicate download write) + Claude Code dispatch with Opus pinned in the spawned argv (claude claude-opus-4-7 --resume <UUID> --print <envelope>)+env text — verbatim operator wording ("We have received new message t communication channel <channel-name>. <classification sentence>. <attachment list>") preceding the <<<HERALD-DISPA v1>>> structured block + <<<HERALD-REPLY>>> parser with action routin default / issue.open / event.emit)+tgram.Adapter.SendReply(c chatID, text, replyToID, attachments) wiring reply_to_mes from the original message_id. --qa-out-dir <path> flag journals every outbound event as JSONL per §107.x. Closing 12 atomic Wave-6 commits T1=a (HERALD_PROJECT_NAME resolver + .env.example), T2=0d55274 (Opus m T3=14626f7 (envelope pre-text), T4=b4018a3 (bot self-filter), T5=d9156f1 OnDocument/OnVoice + sha256 download), T6=a22a054 (pherald listen subcommand), T7=a3b238a (pherald/internal/inbound/ Dispatcher routing), T8=5f92f8a (tgram.SendReply with reply_to_message_id T9=f7913b8 (live closed-loop e2e script tests/test_wave6_live_loop T10a=520e0c6 (--qa-out-dir flag + JSONL journaling middleware), T11= (e2e_bluff_hunt E63-E70 invariants), T12=96c7c6b (Wave 6 mutation gate test_wave6_mutation_meta.sh — 3 paired hermetic mutations). §107 e new e2e invariants E63-E70 land in this commit (all currently SKIP with docme — E63 lifecycle / E64 bot self-filter / E65 attachment sha256 / E66 envelope pre- Opus argv / E68 reply_to_message_id wire-bytes / E69 action routing issu live closed-loop). Conversion SKIP → PASS lands at T10b when the operator exp HERALD_TGRAM_BOT_TOKEN + HERALD_TGRAM_CHAT_ID + HERALD_CL and runs bash tests/test_wave6_live_loop.sh --qa-out-dir HRD-100-<run-id>/. Tag v0.4.0 is DEFERRED to T13b (post-T10b live §107.x) — code-doc closure ships here.

HRD-100 task high

pherald /v1/events Runner wiring (REST API surface via Gin Gonic per spec §32 7-stage event-ingest pipeline lands as 7 concrete structs under pherald/in runner/ (EventParser, IdempotencyChecker, TenantResolver, PolicyGate, SubscriberResolver, ChannelDispatcher, OutcomeRecorder) orchestrated by runner.Runner.Run. pherald/internal/http/events.go exposes HTTP handler; pherald/cmd/pherald/{main,stubs}.go lazily build t JWT verifier inside the serve subcommand so other paths still work without HERALD_PG_DSN/HERALD_AUTH_MODE. HTTP contract: 202 Accepted + Rec on fresh event, 200 + X-Herald-Replay: true on replay, 401 on missing/in 400 on parse failure. §107 evidence: 23/23 runner package tests green (per-stage integration scenarios: happy/duplicate/deny); 6 new e2e invariants E37-E42 (PG skipped when absent — honest SKIP-with-reason per §11.4.3); mutation gate test_wave3_mutation_meta.sh gains M2 (Duplicate=false unconditional breaks), M4 (skip events_processed.Insert → E38/E42 break). M3 SKIP-with-re Wave 3c deny-evaluator e2e. Commits T1..T10: eb9b2f4, 40ad60f, 2fdb7b 4c4af24, 3988114, 5468897, 6ea29a4, be71db1, c7b89a4, (this comm

HRD-016 task middle

ID	Type	Criticality	Title
HRD-011	task	middle	Telegram channel adapter live integration — telebot.v3 vendored (submodules telebot); HealthCheck + Send + Subscribe + outbound_delivery_evidence per live-wired with §107 bluff guards. Closed atomically with operator-session live e 2026-05-22: real bot delivery message_id=5 from @atmosphere_worker_2057253161 (validated via getChat) using the extended wizard CLI's --bot-token (env: HERALD_TGRAM_BOT_TOKEN) + --chat-id (validated via getChat) + --persist + --non-interactive flags. Token→getMe→getChat→sendMessage confirmed end-to-end in commit 140a2f1 session.
HRD-099	task	low	commons_auth/ JWT verifier — HMAC + JWKS hybrid + Gin middleware + cla vendored Herald-internal per §11.4.74 catalogue-check no-match. Catalogue evid docs/catalogue-checks/HRD-099-commons-auth.md. Renumbered HRD-093 (Wave 3a executing-time anchor) in the post-Wave-3a doc-cleanup pas commits dbbea95..0cc6fad keep the old HRD-093 in their commit message
HRD-098	task	middle	sherald /v1/safety_state live — process-local Aggregator + background m (15s tick) + Handler with JWT gate via commons_auth.GinMiddleware. Replaces 501-stub. §107 evidence: e2e E46 (no-auth → 401), E47 (valid HMAC → 200 + current_mem_percent>0 + last_destructive_op=null + uptime_seconds>=1 + oper Paired mutation M5 (zero-mem Aggregator) proves E47 catches the regression. E with-reason — destructive-op trigger awaits HRD-033 body.
HRD-028	task	low	cherald /v1/compliance live — paginated + filter-aware constitution_ surface with JWT gate via commons_auth.GinMiddleware. Backed by ConstitutionStore.ListQuery extended with Since/Until/Offset (commit 6276437) evidence: e2e E43 (no-auth → 401), E44 (valid HMAC + empty tenant → 200 + page=1 + page_size=50). E45 SKIP-with-reason — cross-binary cherald-reads-pl deferred to Wave 3b (Runner not live yet).
HRD-092	task	middle	commons/cli/ shared CLI scaffold (Wave 2) — NewRootCmd + VersionCmd + S (with Middleware hook) + StubCmd + healthz/readyz/metrics handlers. Vendored internal per §11.4.74 catalogue-check no-match. As-built evidence: §44.M of spec docs/catalogue-checks/HRD-092-commons-cli.md.
HRD-093	task	middle	sherald flavor scaffold — System Herald binary serving :24793 with cli.ServeCmd stubs (HRD-033/034/040/046/056) + /v1/safety_state 501 stub → HRD-098.
HRD-094	task	middle	cherald flavor scaffold — Constitution Herald binary serving :24792 with cli.Serve §43 stubs + /v1/compliance 501 stub → HRD-028.
HRD-095	task	low	bherald flavor scaffold — Build Herald CLI-only binary with 3 §43 stubs (HRD-035/041/045).
HRD-096	task	low	rherald flavor scaffold — Release Herald CLI-only binary with 3 §43 stubs (HRD-031/032/045).
HRD-097	task	low	iherald + scherald flavor scaffolds — Incident Herald serving :24794 with /v1/we → HRD-024 + Scheduled-audit Herald CLI-only with 1 §43 stub (HRD-047). Pa iherald has zero §43 stubs.
HRD-012	task	middle	Claude Code dispatcher live integration — claude --resume <UUID> --p <envelope> exec + <<<HERALD-REPLY>>> JSON parse + claude_code_sessions persistence per §33. §107 evidence: Plan 2 Task 6 702b5a3 (live PASS 24.24s — real claude CLI round-trip, structured reply pars Outcome+Summary non-empty) + Task 7 commit 4718c0e (live PASS 36.23s – session_state upsert under HeraldSystemTenant with exact-equality assertions on session_uuid + anchor_path + last_response JSONB round-trip). HRD-085..HRD open for upstream-defined TaskRepository methods not exercised by the Dispatch path.
HRD-010	task	middle	

ID	Type	Criticality	Title
			commons_storage live wiring — pgx pool + RLS-enforcing WithTenantContext (fixed RLS-bypass bug via E14 falsifiability) + 9 migrations + background queue (digital.vasic.background bound via pgxTaskRepository) + Redis ACL (digital.vapherald migrate up/status/down/validate subcommand + 3 new §107 e2e invariants E16) + HRD-085..090 registered for queue-repository stubs
HRD-080	task	low	Refine I6 inheritance-gate invariant from blanket ".gitmodules-forbidden to "constitution/entry in .gitmodules" — paired meta-test test_i6_refinement_meta.sh with 3 anti-bluff subtests. Enables Foundation Helix-stack submodule installs.
HRD-017	task	middle	Propagate Universal §11.4.73 (main-spec versioning + revision discipline) and §11.4.74 (submodule-catalogue-first discovery) into parent constitution Constitution.md + CLAUDE.md + AGENTS.md
HRD-014	task	middle	pherald CLI scaffold — Cobra root + version + 5 stubbed subcommands; pherald version --json returns canonical build info
HRD-013	task	middle	commons_messaging + null:// adapter — full §11.0 Channel contract impl with ri fail_rate/latency/ceiling URL params + 8-case unit test suite
HRD-009b	task	low	commons_prefix module — §8.2 deterministic 3-letter prefix algorithm with Cam + collision-resolution via fnv1a32 + table-driven tests
HRD-009	task	middle	commons module — full §11.0 Go type contract (Channel + Capabilities + Delivery + OutboundMessage + Body + Attachment + Recipient + Action + Priority + Recipient + InboundHandler + InboundEvent + Subscriber + SubscriberAlias + CloudEventEnvelope + TraceContext + Branding + ChannelID + PreferenceSet + ...) + Clock abstraction helper + DefaultBranding factory
HRD-007	task	middle	V3 r3 cross-doc sync + tracking-doc scaffold (Issues/Fixed/Status/Status_Summary/Issues_Summary/Fixed_Summary/CONTINUATION)
HRD-006	task	middle	V3 r2 flavor refinement — 9 flavors × per-channel interaction tables
HRD-005	task	middle	V3 r1 operator-product layer (§31..§36 + §18.2 expansion)
HRD-004	task	middle	V2 r3 — Go type contract closure + operational ops detail
HRD-003	task	middle	V2 r2 — close prose [?] definition gaps + add operational guidance
HRD-002	task	middle	V2 r1 — architectural authoring (CloudEvents/Watermill/OTel/RLS/SLSA/9 flavors)
HRD-001	task	middle	V1 — initial MVP specification + Review section + Recommendations